

Java questions:

1. == vs equals() vs hashCode()
2. String pool and String immutability
3. Immutability and how to design an immutable class
4. final, finally, finalize
5. Overloading vs overriding
6. **Abstract class vs interface**
7. **Default/static methods in interfaces**
8. Exception hierarchy
9. Checked vs unchecked exceptions
10. Serialization/deserialization
11. **transient, volatile, static keyword**
12. **Records, sealed classes, pattern matching**
13. Covariant return types
14. String vs StringBuilder vs StringBuffer
15. Composition vs inheritance
16. try-with-resources
17. AutoCloseable vs Closeable
18. Shallow copy vs deep copy
19. Generics and type erasure
20. PECS: ? extends vs ? super
21. **Comparator vs Comparable**
22. How does HashSet work internally?
23. Exception handling with method overriding
24. Difference between wait, sleep and yield
25. Future vs completable future
26. Java garbage collectors
27. Association vs Aggregation vs Composition
28. Mock vs InjectMock
29. Mock vs Spy
30. doReturn vs thenReturn
31. How to mock void method?
32. Constructor injection vs Setter injection
33. Marker interface

Additional questions:

34. Collection:

- **What is the internal working of HashMap?**
- **HashMap And synchronized vs ConcurrentHashMap Difference?**
- How can you put all elements into the same bucket in HashMap?
- Hashcode and Equals Contracts
- How many default buckets and load factory does HashMap have?

- Why do we need multiple buckets in HashMap?
- Fail Safe and Fail-Fast Iterators?
- ConcurrentHashMap bucket level locking
- How do we achieve ConcurrentModification . Inside the map I want to add new element will it be iterator?
- Array vs ArrayList
- If an array has fixed size, why use array instead of ArrayList?
- map vs flatmap
- Difference between comparator and comparable

35. OOPS

- Overriding with static method -> Method hiding
- Overriding with some scenario like exception, static, final keyword
- A functional interface has how many abstract methods?
- Functional Interface and Lambda differences?
- Predicate, supplier, consumer, function
- Diamond Problem: Consider one interesting scenario: You have two interfaces, Interface1 and Interface2 both having default method hello() and one class is implementing these 2 interfaces without giving implementation to this default method. What is the problem here?

36. What is a Singleton class?

- Singleton class in multithreaded env -> Double check locking
- If a Singleton class implements Serializable, what is the impact?
- How can you resolve the serialization issue in Singleton?

37. Immutability in java

- Why are Strings Immutable?
- How to create a immutable class, Create an immutable class Employee having Address as instance variable

38. Why are wait(), notify(), and notifyAll() defined in Object class instead of Thread class?

39. How to throw an exception in JUnit? How will you write?

40. Features of Java 8,11,17,21, 25: Top 2 imp features

- What is a Java record? Are records immutable?
- What is an optional class?
- What is sealed class?
- Virtual thread

41. Why is the main method static?

- Java pass-by-value ?
- How many ways can a Java object be created?
- What is the superclass of all classes in Java? What are its methods? Java extends two classes -> Object class + class A

42. What are annotations in Java? Can you create your own annotations in Java?

43. What built-in annotations are provided by Java?

44. Exceptional handling:

- Difference between throw and throws keyword?
- What is try with resources ?

45. What is the Java Memory Model?
 - What can cause OutOfMemory Error? How to fix it : JProfiler, VirtualVM
 - What is a memory leak, and how would you prevent it in Java?
46. Iterate over a String without using predefined methods
47. Reactive programming : MOno and Flux uses Non blocking IO
48. Reverse a string using java stream API
49. What is Marker interface
50. Garbage collector: -> V Imp
 - CMS vs G1 vs ZGC
 - Mark sweep Algo
 - Type of GC usecases usage Eden memory
51. Testing:
 - Difference between Unit Testing and Integration Testing?

Multithreading questions:

1. Write a program where one thread prints odd numbers and another prints even numbers.
2. What is ThreadLocal?
3. volatile vs Atomic
4. CountdownLatch and CyclicBarrier
5. Explain completable future -> runAsync SupplyAsync
6. Object level lock vs Class level lock
7. Reentrant lock
8. How to achieve thread safety in a multi threaded environment?
9. Explain Executor service and explain different type of executors
 - newCachedThreadPool() vs newFixedThreadPool()
10. Explain Deadlocks
11. Difference between sleep, wait and join
12. Implement blocking queue OR Pub sub using wait and notify -> LLD
13. Fork join poll and work stealing
14. Write a Java program to print a sequence of numbers up to N using 3 threads. For example, if we want to print a sequence of numbers up to 10 then it'll look like this:

THREAD-1 : 1

THREAD-2 : 2

THREAD-3 : 3

THREAD-1 : 4

THREAD-1 : 10

Stream ::

1) Give the count of every Character in this string using Java stream api.

```
String str="abcdabcghsk";
```

ans:

```
System.out.println(Arrays.stream(str.split(""))  
    .collect(Collectors.groupingBy(Function.identity(),  
    Collectors.counting())));
```

Or

```
System.out.println(str.chars().mapToObj(c -> (char) c)  
    .collect(Collectors.groupingBy(Function.identity(),  
    Collectors.counting())));
```

2) Merge the below two lists using Java stream api.

```
List<Integer> list1= Arrays.asList(1,2,3,4);
```

```
List<Integer> list2= Arrays.asList(5,6,7,8);
```

ans:

```
List<Integer> mergedList = Stream.concat(list1.stream(), list2.stream())  
    .collect(Collectors.toList());  
System.out.println(mergedList); // Output: [1, 2, 3, 4, 5, 6, 7, 8]
```

Or

```
Stream.of(list1, list2).flatMap(List::stream).toList();
```

3) Write a code to reverse the below string using Java stream API.

```
String str = "Stream";
```

ans:

```
System.out.println(IntStream.range(0, str.length()).mapToObj(i ->  
    str.charAt(str.length() - 1 -  
    i)).map(Object::toString).collect(Collectors.joining()));
```

Or

```
System.out.println(str.chars().mapToObj(c -> String.valueOf((char)
```

```
c)).reduce("", (s, c) -> c + s));
```

4) Delete all the occurrences of 'c' from the given String and print the String.

```
String s = "occurance"
```

```
ans: String ans = s.chars().mapToObj(c -> (char) c).filter(ch -> ch != 'c')
.map(String::valueOf).collect(Collectors.joining());
System.out.println(ans);
```

Output: "ourane"

5) List all the duplicate Characters in the String.

```
String s = "programming";
```

```
ans: List<Character> duplicate = s.chars().mapToObj(c -> (char) c)
.collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
.entrySet().stream().filter(entry -> entry.getValue() > 1)
.map(Map.Entry::getKey).collect(Collectors.toList());
System.out.println(duplicate);
```

6) Given a list of strings, count how many times each word appears using Java 8.

```
List<String> words = Arrays.asList("apple", "banana", "apple", "orange", "banana");
```

```
Map<String, Long> wordCount =
words.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

```
System.out.println(wordCount);
```

Output:

```
apple → 2
banana → 2
orange → 1
```

59) Given a sentence as a String, return the second-highest distinct word length present in the sentence.

```
String sentence = "I am working in motivated organization"
```

```
ans: int result = Arrays.stream(sentence.split(" "))
.map(String::length)
.distinct()
.sorted(Comparator.reverseOrder())
.skip(1)
.findFirst()
.orElse(0);
```

```
System.out.println(result);
```

Output: 9

60) Find out the even Integers from the given list using stream API.

```
List<String> list = Arrays.asList("one", "5", "ab12", "abc", "8", "20");
```

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
```

```
public class EvenNumbers {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("one", "5", "ab12", "abc", "8", "20");

        List<Integer> evenIntegers = list.stream()
            .filter(EvenNumbers::isNumeric) // Keep only valid number strings
            .map(Integer::parseInt)         // Convert string to integer
            .filter(n -> n % 2 == 0)        // Keep only even numbers
            .collect(Collectors.toList()); // Collect results into a list

        System.out.println(evenIntegers); // Output: [8, 20]
    }
}
```

```
// Helper method to check if a string is a valid integer
public static boolean isNumeric(String str) {
    if (str == null) return false;
    try {
        Integer.parseInt(str);
        return true;
    } catch (NumberFormatException e) {
        return false;
    }
}
}
```

Or

```
public class EvenNumbers {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("one", "5", "ab12", "abc", "8", "20");

        List<Integer> evenIntegers = list.stream()
            .filter(s -> s.matches("\\d+")) // Keep only valid number strings
            .map(Integer::parseInt)         // Convert string to integer
            .filter(n -> n % 2 == 0)        // Keep only even numbers
            .collect(Collectors.toList()); // Collect results into a list
    }
}
```

```

        System.out.println(evenIntegers); // Output: [8, 20]
    }
}

```

61) Assume that you have a list of names and your list contains a mix of names that are ending with "lal" and not ending with lal as well.

```
List<String> list = Arrays.asList(Ram,Vinitha,Mohanlal,Jawaharlal);
```

a. Give me a Java code using stream api to list the names ending with "lal".

```
ans: List<String> result = list.stream()
    .filter(name -> name.endsWith("lal"))
    .toList();
```

b. Give me a Java code using stream api to list the names not ending with "lal".

```
ans: List<String> result = list.stream()
    .filter(name -> !name.endsWith("lal"))
    .toList();
```

c. If I add null as input to the list i.e. {Ram,Vinitha,Mohanlal,Jawaharlal,null} - the code must not throw a null pointer exception and as well display names not ending with "lal".

```
ans: List<String> result = list.stream()
    .filter(name -> name != null && !name.endsWith("lal"))
    .toList();
```

47) Second largest number using Java stream API:

```
List<Integer> numbers = Arrays.asList(10, 5, 15, 20, 8);
```

```
ans: Integer secondLargest = numbers.stream()
    .distinct() // Remove duplicates
    .sorted(Comparator.reverseOrder()) // Sort in descending order
    .skip(1) // Skip the largest number
    .findFirst()
    .orElse(null); // Handle empty list
```

48) Unique characters of a string using Java stream API:

```
String input = "hello world";
```

```
ans: Set<Character> uniqueChars = input.chars()
```

```
.mapToObj(c -> (char) c)
.collect(Collectors.toSet());
```

49) Convert List<Employee> into Map<empId, empName> using stream API.

ans: `Map<Integer, String> employeeMap = employees.stream()
.collect(Collectors.toMap(Employee::getEmpId, Employee::getEmpName));`

50) Convert List<Employee> into Map<empId, empName> using stream API and also maintain insertion order.

ans:

```
Map<Integer, String> employeeMap =
employees.stream().collect(Collectors.toMap(Employee::getEmpId,
Employee::getEmpName, (v1, v2) -> v1, LinkedHashMap::new));
```

51) Give vehicle name who has travelled maximum in august month. Here, there is one vehicle table having vehicle id, name and model. Other table is journey having vehicle id, date of journey and distance travelled.

ans:

```
String maxDistanceVehicle = journeyList.stream()
    .filter(j -> j.getDateOfJourney().getMonthValue() == 8)
    .collect(Collectors.groupingBy(Journey::getVehicleId,
Collectors.summingDouble(Journey::getDistanceTravelled)))
    .entrySet()
    .stream()
    .max(Map.Entry.comparingByValue())
    .map(entry -> vehicleList.stream()
        .filter(v -> v.getId() == entry.getKey())
        .findFirst()
        .get().getName())
    .orElse("No vehicles travelled in August");
```

52) Return list of duplicated integers from arraylist of integers.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 2, 4, 5, 3, 6);
```

ans: `List<Integer> duplicates = nums.stream()
 .filter(n -> nums.stream().filter(m -> m == n).count() > 1)
 .distinct()
 .collect(Collectors.toList());`

53) Can we use stream() method directly for an array of int?

ans: No, need to use **Arrays.stream(<arr_name>)**

54) Return list of employees whose name startsWith letter 'A' and city is Mumbai or Pune.

ans:

```
Predicate<Employee> nameStartsWithA = e -> e.getName().startsWith("A");  
Predicate<Employee> cityIsMumbaiOrPune = e -> e.getCity().equalsIgnoreCase("Mumbai") ||  
e.getCity().equalsIgnoreCase("Pune");
```

```
List<Employee> filteredEmployees = employees.stream()  
    .filter(nameStartsWithA.and(cityIsMumbaiOrPune))  
    .collect(Collectors.toList());
```